ASAAN INNOVATIONS

INTERN AGENT BUILD

ASSIGNMENT 2

Banking Agents Built on Your Fine-Tuned Models

Task Brief, Technical Guidance, and Evaluation Criteria

Prepared for the Intern Cohort

July 2026

Asaan Innovations

Internal Task Brief. Technical Documentation.

Intern Assignment 2: Banking Agents Built on Your Fine-Tuned Models

Prepared for the Intern Cohort, July 2026

1. Purpose and Scope

This is the next assignment following the fine-tuning prototype sprint you have just submitted. The goal now is to build an agent, not just a model that answers a prompt, that uses the LLM you fine-tuned in the previous assignment as its core reasoning component, applied to a real banking use case.

Deadline: Thursday, July 30, 2026, 11:59 PM Pakistan Standard Time. This is a firm deadline.

As with the previous assignment, this document is a guide, not a fixed script. You are expected to make real design decisions, choice of use case, agent architecture, how the fine-tuned model is used within it, and to justify them. Creativity in problem selection and agent design matters as much as getting something working.

2. Assignment Overview

Item	Detail
Submissions	Individual. Each intern works and submits on their own
Deadline	Thursday, July 30, 2026, 11:59 PM Pakistan Standard Time
Domain	Banking only. Not finance in general, the use case must be something a bank specifically would want automated
Core requirement	The agent must use the LLM you fine-tuned in the previous assignment as its reasoning engine, either as the primary decision maker or as a specialist component within a larger orchestration
Output	A working agent prototype, a short demo showing it handling a few example cases, and a brief written summary

3. What Makes This an Agent

A single prompt in, single answer out is not an agent, that was effectively the previous assignment. An agent is expected to show at least some of the following:

- Multi-step reasoning: breaking a case into steps rather than producing one final answer directly
- Tool use: calling a function, a lookup, a calculation, or a rule check as part of reaching its answer, rather than relying purely on the model's own generated text
- Decisions that lead to different next actions: for example escalate versus auto-resolve, approve versus refer for manual review, request more information versus proceed
- Some ability to handle a case that unfolds over more than one exchange, if your use case naturally involves that

You do not need all of these at once, but a submission that is just your fine-tuned model answering a single prompt, with no tool calls, no decision branching, and no structure around it, does not meet the bar for this assignment.

4. Use Case Selection

Choose one real banking use case for your agent to handle. It can extend the task you fine-tuned on in the previous assignment, or be a new one, but it must stay within banking specifically. Some starting ideas, or invent your own:

- A loan application agent that gathers the necessary details, checks them against rules, and produces an approve, decline, or refer-for-review decision with reasoning
- A fraud alert triage agent that reviews a flagged transaction or account activity summary, decides whether it looks genuinely suspicious, and recommends a next action
- A customer complaint resolution agent that classifies an incoming complaint, checks it against a policy or compliance rule set, and drafts a response or escalation
- A transaction dispute investigation agent that walks through the facts of a disputed charge and reaches a recommendation
- An account opening or KYC verification agent that checks submitted information against a set of requirements and flags gaps or concerns
- A collections follow-up agent that reviews an overdue account's history and decides on next contact strategy within policy limits

Pick something specific and well scoped rather than a broad, vague banking assistant. A narrow use case you can actually demonstrate working end to end is worth more than a broad one that only partly works.

5. Agent Architecture Guidance

Your fine-tuned model can play one of two roles, both are valid, pick whichever fits your use case and justify it:

- As the primary reasoning engine: your fine-tuned model itself decides what to do at each step, including when to call a tool and how to interpret the result
- As a specialist component: a broader orchestration layer, which could be plain Python control flow or another LLM, handles the overall flow and calls your fine-tuned model for the specific judgment or classification step it was trained for

Tools do not need to be real external systems for this prototype. A mock function that simulates a database lookup, a credit check, a policy rule, or a calculation is entirely acceptable, the point is to demonstrate the agent pattern (the model's output feeding into a decision or action, not just producing text) rather than to integrate live banking systems.

Any orchestration approach is acceptable: a simple custom loop in Python, a framework such as LangChain or LlamaIndex, or something you put together yourself. Simpler and working is better than elaborate and broken given the timeline.

6. Tools and Environment

Continue using the fine-tuned model checkpoint from the previous assignment. If it needs light adjustment to fit this use case, that is fine, but a full retraining cycle is not expected given the timeline.

Inference can continue to run on Kaggle (2x T4) or Colab (1x T4) free tier as before, or locally or via CPU if your model is small enough to make that practical. Orchestration logic (the agent loop, tool calls, decision logic) can run in the same notebook or as a separate script alongside it, whichever is easier to demo.

7. Timeline

Window	Focus
Now through Thursday afternoon	Pick your use case, sketch the agent's steps and decision points, get the fine-tuned model wired into a basic version of the flow
Thursday afternoon into evening	Add tool calls and decision branching, test against a handful of example cases, fix rough edges, write up your summary
Thursday, 11:59 PM PKT	Submission deadline

8. Daily Updates

Send a daily update before 11:59 PM, posted as a brief paragraph on the WhatsApp group. Cover what you worked on, your use case and agent architecture, and anything blocking you. Keep it short, the point is surfacing problems early enough that they can still be addressed before the Thursday deadline.

9. Deliverables

- Your agent's code, as a notebook, script, or small repository, shared as a link or file
- A short demo: a transcript or short recording showing your agent handling two or three example cases from input to final output, including any intermediate steps or tool calls it took
- A brief written summary: the use case, why it matters for a bank, your agent's architecture and why you designed it that way, which tools were real versus mocked, and current limitations
- Your daily updates as described above

10. Evaluation Criteria

This assignment is judged on the following, roughly in order of importance:

- Relevance and specificity of the banking use case chosen
- Quality of the agent architecture, does it genuinely reason and act in steps, rather than wrapping a single prompt
- How well the fine-tuned model from the previous assignment is put to use within the agent
- Quality and clarity of the demo, can someone else follow what the agent did and why
- Clarity of the written summary and honesty about limitations
- Completion within the deadline, a simple agent that is finished and demonstrated is worth more than an ambitious one that is not

11. Assumptions and Notes

- Tools called by the agent may be mocked or simulated for this prototype, live integration with real banking systems or external APIs is not required
- The fine-tuned model from the previous assignment is the expected starting point; light adjustments are fine, a full retraining cycle is not expected given the timeline
- Free tier GPU availability is not guaranteed and fluctuates with demand, build in buffer time rather than assuming a GPU will be available exactly when needed
- As before, this document is a guide, not a fixed script, creativity in use case selection and agent design is expected and encouraged